

Aprendizaje de Máquina

Sesión 13: Reducción de dimensiones

Angel Rodríguez

Examen

- El primer examen será el **jueves 24 de Septiembre**.
- En

Actividades de aprendizaje autónomo -> Guías para exámenes -> Guía para el parcial #1

encontrarán (antes de la siguiente clase) una guía para el examen.

Contenido: Capítulos 1 y 2.

Equipos para el proyecto final

RMSE — Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- Es simplemente la raíz cuadrada del MSE:

$$RMSE = \sqrt{MSE}.$$

- Al igual que MSE, **penaliza fuertemente los errores grandes**.
- A diferencia de MSE, está expresado en las **mismas unidades que y**.

¿Cuándo utilizar RMSE?

- Cuando queremos penalizar errores grandes pero mantener una métrica **interpretable en las unidades originales**.
- Es útil cuando errores grandes son especialmente indeseables.

MAE vs. MSE vs. RMSE

Métrica	Penalización	Unidades	Útil cuando...
MAE	Lineal	Originales	Hay outliers
MSE	Cuadrática	Al cuadrado	Importan errores grandes
RMSE	Cuadrática	Originales	Importan errores grandes e interpretación

Nota:

No existe una métrica universalmente “mejor”.

La métrica debe reflejar **qué tipo de error queremos penalizar**.

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Mide la **proporción total de predicciones correctas**.

¿Cuándo es útil?

- Cuando las clases están **razonablemente balanceadas**.
- Cuando FP y FN tienen costos similares.

Cuidado con clases desbalanceadas

Si el 99 % de las observaciones pertenece a una clase, un modelo que siempre prediga esa clase tendría

$$Accuracy = 99\%$$

aunque sea inútil para detectar la clase minoritaria.

Precision

$$Precision = \frac{TP}{TP + FP}$$

Responde a la pregunta:

**De todo lo que predije como positivo,
¿qué proporción realmente era positiva?**

Conviene priorizar Precision cuando los falsos positivos (FP) son especialmente costosos.

Por ejemplo:

- Un filtro que bloquea correos legítimos como spam.
- Una alarma que genera intervenciones costosas cada vez que predice un evento.

Recall (o sensibilidad)

$$Recall = \frac{TP}{TP + FN}$$

Responde a la pregunta:

**De todos los positivos reales,
¿qué proporción logré detectar?**

Conviene priorizar Recall cuando los falsos negativos (FN) son especialmente costosos.

Por ejemplo:

- Detección de fraude.
- Detección de defectos en productos.
- Sistemas de alerta donde es importante no omitir eventos relevantes.

Precision vs. Recall

Existe frecuentemente un **trade-off** entre Precision y Recall.

Priorizar Precision

↓ *FP*

- Queremos estar seguros antes de predecir positivo.
- Los falsos positivos son costosos.

Priorizar Recall

↓ *FN*

- Queremos encontrar tantos positivos como sea posible.
- Los falsos negativos son costosos.

Pregunta clave

¿Qué error es más costoso para nuestro problema: un **falso positivo** o un **falso negativo**?

F1-score

Precision y Recall pueden combinarse utilizando su **media armónica**:

$$F_1 = 2 \frac{\textit{Precision} \cdot \textit{Recall}}{\textit{Precision} + \textit{Recall}}$$

Equivalentemente,

$$F_1 = \frac{2TP}{2TP + FP + FN}.$$

¿Cuándo utilizar F1-score?

- Cuando las clases están **desbalanceadas**.
- Cuando nos importan tanto FP como FN.
- Cuando queremos una sola métrica que combine **Precision y Recall**.

¿Qué métrica de clasificación utilizar?

Situación	Métrica
Clases balanceadas y errores similares	Accuracy
Los falsos positivos son costosos	Precision
Los falsos negativos son costosos	Recall
Queremos balancear Precision y Recall	F1-score
Clases desbalanceadas	Precision / Recall / F1

En resumen

La métrica adecuada depende del **objetivo del problema** y del **costo de cada tipo de error**.

Ejemplos

Métrica	Ejemplo	¿Por qué conviene?
Accuracy	Perros vs. gatos	Clases balanceadas
Precision	Filtro de spam	Evitar falsos positivos
Recall	Detector de incendios	Evitar falsos negativos
F1-score	Detección de fraude	Balancear precision y recall

Idea clave

La métrica adecuada depende del **costo de cada tipo de error**.

Accuracy: aciertos generales

Pregunta que responde:

De todas mis predicciones, ¿qué porcentaje fue correcto?

Ejemplo: perros vs. gatos

- Tenemos imágenes de perros y gatos.
- Las clases están aproximadamente balanceadas.
- Equivocarse con un perro o con un gato tiene un costo similar.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Las clases están balanceadas y ambos tipos de error tienen costos parecidos.

Precision: confiar en los positivos

Pregunta que responde:

Cuando el modelo dice “positivo”, ¿qué tan seguido tiene razón?

Ejemplo: filtro de spam

- Positivo = el correo es spam.
- Un falso positivo significa marcar un correo legítimo como spam.
- Si el sistema elimina automáticamente correos, un falso positivo puede ser muy costoso.

$$Precision = \frac{TP}{TP + FP}$$

Queremos evitar **falsos positivos**.

Recall: encontrar los positivos reales

Pregunta que responde:

De todos los positivos reales, ¿cuántos encontró el modelo?

Ejemplo: detector de incendios

- Positivo = hay incendio.
- Un falso negativo significa que sí hay incendio, pero el sistema no lo detecta.
- Es preferible tolerar algunas falsas alarmas antes que omitir un incendio real.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Queremos evitar **falsos negativos**.

F1-score: equilibrio entre precision y recall

Pregunta que responde:

¿Qué tan bien equilibramos precision y recall?

Ejemplo: detección de fraude

- Positivo = transacción fraudulenta.
- Los fraudes suelen ser poco frecuentes.
- Accuracy puede ser engañosa si casi todo es “no fraude”.
- Queremos detectar fraudes, pero sin bloquear demasiadas transacciones legítimas.

$$F_1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

Resumen intuitivo

Métrica	Pregunta intuitiva
Accuracy	¿Qué tanto acierto en general?
Precision	Cuando digo positivo, ¿qué tan seguido acierto?
Recall	De los positivos reales, ¿cuántos encontré?
F1-score	¿Qué tan bien balanceo precision y recall?

Regla práctica

Precision se preocupa por reducir falsos positivos.

Recall se preocupa por reducir falsos negativos.

F1-score busca un balance entre ambas.

¿Por qué reducir dimensiones?

Muchos conjuntos de datos tienen miles o millones de características.

Beneficios

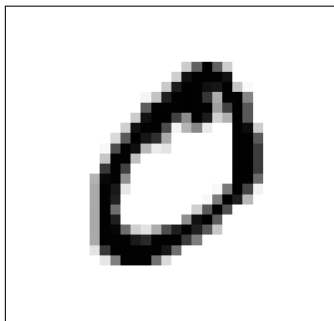
- Entrenamiento más rápido.
- Menor uso de memoria y disco.
- Menos ruido o redundancia.
- Visualización en 2D o 3D.

Riesgos

- Se puede perder información útil.
- Puede empeorar el desempeño.
- Agrega un paso extra al pipeline.
- No siempre ayuda a redes neuronales profundas.

Ejemplo: píxeles redundantes

En imágenes como MNIST, no todos los píxeles aportan la misma información.



La maldición de la dimensionalidad

Nuestra intuición de 2D/3D falla en espacios de muchas dimensiones.

- En dimensiones altas, los puntos aleatorios tienden a estar lejos entre sí.
- Los datos se vuelven más **dispersos**.
- Métodos basados en distancia, como k-NN, pueden volverse menos confiables.
- Para mantener la misma densidad, el número de muestras necesarias crece exponencialmente.

Idea clave

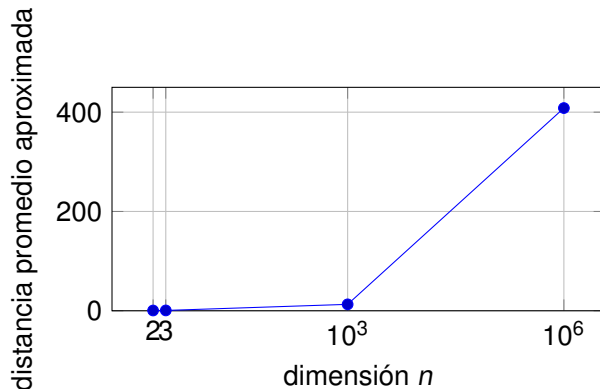
Más dimensiones no significan automáticamente más información útil; también pueden significar más espacio vacío.

Distancias en alta dimensión

- Si tomamos dos puntos aleatorios en un cuadrado unitario, la distancia entre 2 puntos aleatorios en promedio será de 0.52.
- Si tomamos dos puntos aleatorios en un cubo unitario, la distancia entre 2 puntos aleatorios en promedio será de 0.66.
- ¿Y para un hipercubo de 1 millón de dimensiones?

Distancias en alta dimensión

Si tomamos dos puntos aleatorios en un hipercubo unitario:



La distancia típica crece como una función de la dimensión, incluso cuando cada coordenada está entre 0 y 1.

Distancia entre puntos en dimensión alta

Sean dos puntos aleatorios independientes

$$x, y \sim U([0, 1]^d).$$

La distancia euclidiana al cuadrado es

$$D^2 = \|x - y\|^2 = \sum_{i=1}^d (x_i - y_i)^2.$$

Distancia entre puntos en dimensión alta

Sean dos puntos aleatorios independientes

$$x, y \sim U([0, 1]^d).$$

La distancia euclidiana al cuadrado es

$$D^2 = \|x - y\|^2 = \sum_{i=1}^d (x_i - y_i)^2.$$

Para cada coordenada,

$$\begin{aligned}\mathbb{E}[(x_i - y_i)^2] &= \mathbb{E}[x_i^2] - 2\mathbb{E}[x_i y_i] + \mathbb{E}[y_i^2] \\ &= \frac{1}{3} - 2\left(\frac{1}{2} \frac{1}{2}\right) + \frac{1}{3} \\ &= \boxed{\frac{1}{6}}.\end{aligned}$$

Distancia entre puntos en dimensión alta

Por linealidad de la esperanza,

$$\mathbb{E}[D^2] = \sum_{i=1}^d \mathbb{E}[(x_i - y_i)^2] = \boxed{\frac{d}{6}}.$$

Distancia entre puntos en dimensión alta

Por linealidad de la esperanza,

$$\mathbb{E}[D^2] = \sum_{i=1}^d \mathbb{E}[(x_i - y_i)^2] = \boxed{\frac{d}{6}}.$$

Por tanto, la distancia típica puede aproximarse por

$$\boxed{D_{tpica} \approx \sqrt{\mathbb{E}[D^2]} = \sqrt{\frac{d}{6}}}.$$

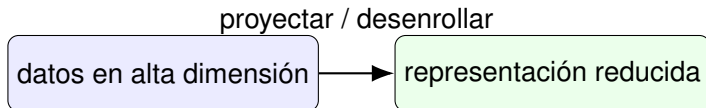
Dos enfoques principales

1. Proyección

- Los datos están cerca de un subespacio de menor dimensión.
- Se proyectan sobre ese subespacio.
- Ejemplo: PCA.

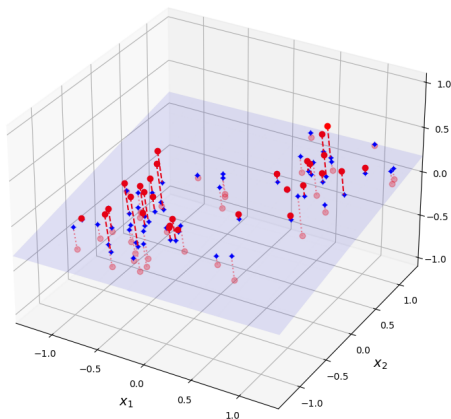
2. Manifold learning

- Los datos viven cerca de una variedad no lineal.
- Se busca “desenrollar” o parametrizar esa estructura.
- Ejemplo: LLE, Isomap, t-SNE, UMAP.



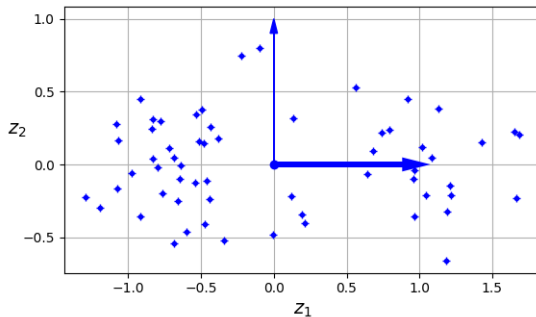
Proyección

En muchos problemas reales, las muestras no llenan todo el espacio.



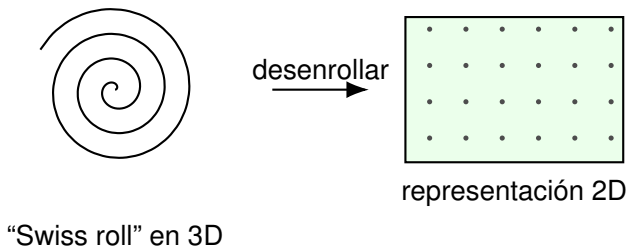
Proyección

Proyectar reduce la dimensión:

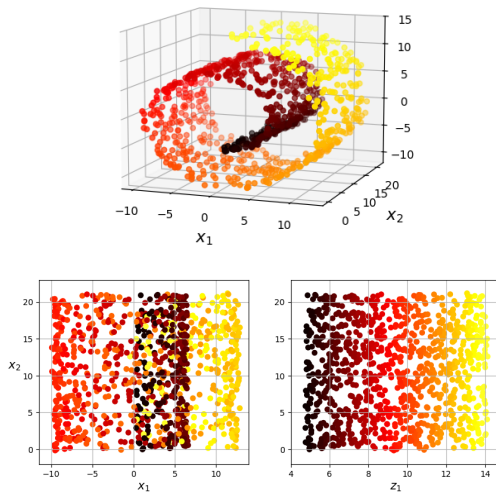


Manifold learning

Una variedad de baja dimensión puede estar doblada en un espacio de mayor dimensión.



"Swiss Roll"



Hipótesis de la variedad

Una **variedad** de dimensión d es un subconjunto de un espacio de dimensión n (con $d < n$), que localmente parece un hiperplano de dimensión d .

La **hipótesis de la variedad** dice que muchos datos reales de alta dimensión están cerca de una estructura de menor dimensión.

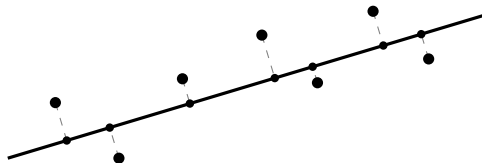
Ejemplo: una imagen de 28×28 tiene 784 píxeles, pero no cualquier arreglo de 784 números parece un dígito manuscrito.

Advertencia

Reducir dimensiones suele acelerar el entrenamiento, pero no garantiza que el problema supervisado sea más simple. Depende del conjunto de datos y de la tarea.

PCA también minimiza error de reconstrucción

Elegir la dirección de máxima varianza equivale a elegir la proyección que minimiza la distancia cuadrática media entre los datos originales y sus proyecciones.



PCA elige el subespacio que conserva más varianza

PCA (teoría)¹

Tenemos datos

$$x_1, \dots, x_N \in \mathbb{R}^D.$$

Queremos representarlos en un espacio de menor dimensión:

$$M < D.$$

La idea de PCA es encontrar direcciones sobre las cuales los datos tengan la mayor variabilidad posible.

Intuición

La primera componente principal es la dirección donde las proyecciones de los datos quedan más “extendidas”.

¹ Pattern Recognition and Machine Learning - Christopher M. Bishop

Datos centrados

Primero calculamos la media muestral:

$$\bar{x} = \frac{1}{N} \sum_{n=1}^N x_n.$$

Después trabajamos con los datos centrados:

$$x_n - \bar{x}.$$

La matriz de covarianza muestral es

$$S = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(x_n - \bar{x})^T$$

Proyección en una dimensión

Buscamos una dirección

$$u_1 \in \mathbb{R}^D$$

sobre la cual proyectar los datos.

Tomamos u_1 como vector unitario:

$$u_1^\top u_1 = 1$$

La proyección del dato x_n sobre esta dirección es

$$z_{n1} = u_1^\top (x_n - \bar{x}).$$

Así, cada punto de $\mathbb{R}^D$ se resume con un solo número.

Varianza proyectada

Queremos que las proyecciones tengan máxima varianza.

La varianza de los datos proyectados es

$$\frac{1}{N} \sum_{n=1}^N \left[u_1^T (x_n - \bar{x}) \right]^2.$$

Esta expresión se puede escribir como

$$u_1^T S u_1$$

Por lo tanto, el problema de PCA en una dimensión es

$$\boxed{\begin{array}{l} \text{máx} \\ u_1 \end{array} u_1^T S u_1 \quad \text{sujeto a} \quad u_1^T u_1 = 1.}$$

¿Por qué necesitamos una restricción?

Si no imponemos ninguna restricción, podríamos hacer

$$\|u_1\| \rightarrow \infty$$

y entonces

$$u_1^T S u_1$$

crecería artificialmente.

Por eso imponemos

$$u_1^T u_1 = 1.$$

Idea

No nos interesa la longitud de u_1 .

Nos interesa únicamente su **dirección**.

Multiplicadores de Lagrange

Queremos maximizar

$$u_1^T S u_1$$

sujeto a

$$u_1^T u_1 = 1.$$

Definimos el Lagrangiano:

$$\mathcal{L}(u_1, \lambda_1) = u_1^T S u_1 + \lambda_1 (1 - u_1^T u_1).$$

Derivando respecto a u_1 e igualando a cero:

$$2S u_1 - 2\lambda_1 u_1 = 0.$$

Aparece el problema de eigenvalores

De la condición de primer orden obtenemos

$$Su_1 = \lambda_1 u_1.$$

Esto significa que u_1 debe ser un **eigenvector** de la matriz de covarianza S .

Además, como $u_1^T u_1 = 1$, entonces

$$u_1^T Su_1 = \lambda_1.$$

Conclusión

La varianza proyectada es igual al eigenvalor correspondiente.

Primera componente principal

Como queremos maximizar la varianza proyectada,

$$u_1^T S u_1 = \lambda_1,$$

debemos elegir el eigenvector asociado al eigenvalor más grande.

$u_1 = \text{eigenvector de } S \text{ asociado al mayor eigenvalor}$

Primera componente principal

La primera componente principal es la dirección en la que los datos tienen máxima varianza.

Más componentes principales

La segunda componente principal se obtiene buscando otra dirección que maximice la varianza, pero que sea ortogonal a la primera:

$$u_2^\top u_1 = 0.$$

En general, las componentes principales son

$$u_1, u_2, \dots, u_M,$$

donde cada u_j es eigenvector de S . Se eligen en orden descendente de eigenvalores:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_M.$$

Interpretación: Cada nueva componente captura la mayor varianza restante, sujeta a ser ortogonal a las anteriores.

Representación en baja dimensión

Agrupamos las primeras M componentes principales en la matriz

$$U_M = \begin{bmatrix} u_1 & u_2 & \cdots & u_M \end{bmatrix}.$$

La representación reducida de x_n es

$$z_n = U_M^T(x_n - \bar{x})$$

donde

$$z_n \in \mathbb{R}^M.$$

Así pasamos de D dimensiones a M dimensiones.

Resumen

1. Centramos los datos.
2. Calculamos la matriz de covarianza:

$$S = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(x_n - \bar{x})^T.$$

3. Buscamos direcciones de máxima varianza:

$$\max_u u^T S u \quad \text{s.a.} \quad u^T u = 1.$$

4. Esto lleva al problema:

$$Su = \lambda u.$$

5. Tomamos los eigenvectores con eigenvalores más grandes.

PCA con descomposición en valores singulares (SVD)

Sin embargo, hay una manera más eficiente para calcular las componentes principales. Se centra la matriz de datos y se aplica SVD:

$$X_{\text{centrada}} = U\Sigma V^T.$$

Las columnas de V contienen las direcciones principales.

Ejemplo

```
import numpy as np
import matplotlib.pyplot as plt

angle = np.pi / 5
stretch = 5
m = 200

rng = np.random.default_rng(seed=3)
X_line = rng.standard_normal((m, 2)) / 10
X_line = X_line @ np.array([[stretch, 0], [0, 1]]) # stretch
X_line = X_line @ [[np.cos(angle), np.sin(angle)],
                    [np.sin(angle), np.cos(angle)]] # rotate

plt.scatter(X_line[:,0],X_line[:,1])

X_centered = X_line - X_line.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)

c1 = Vt[0] # primera componente principal
c2 = Vt[1] # segunda componente principal
```

Proyectar a d dimensiones

Si W_d contiene las primeras d componentes principales, entonces

$$X_{d\text{-proj}} = XW_d.$$

- X : matriz original centrada.
- W_d : matriz con las primeras d direcciones principales.
- $X_{d\text{-proj}}$: representación reducida.

Interpretación

Cada nueva característica es una combinación lineal de las características originales.

Ejemplo

```
X_proy = X_centered@Vt[:1].T  
plt.scatter(X_proy,np.zeros(m))
```

PCA en Scikit-Learn

```
from sklearn.decomposition import PCA  
  
pca = PCA(n_components=1)  
X1D = pca.fit_transform(X_line)
```

Scikit-Learn centra los datos automáticamente.

```
pca.components_ # componentes principales  
pca.explained_variance_ratio_ # varianza explicada por componente
```

Varianza explicada

La razón de varianza explicada indica qué proporción de la variabilidad total se conserva en cada componente.

Ejemplo:

`explained_variance_ratio_ = [0,82, 0,11, 0,07].`

- PC1 conserva aproximadamente 82 % de la varianza.
- PC2 conserva aproximadamente 11 %.
- PC3 conserva aproximadamente 7 %.

Elegir el número de dimensiones

Una práctica común es conservar, por ejemplo, 95 % de la varianza.

```
from sklearn.decomposition import PCA
import numpy as np

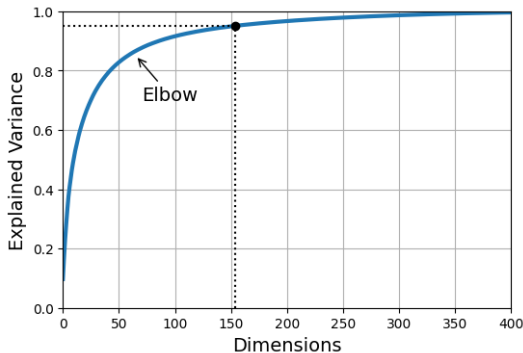
pca = PCA()
pca.fit(X_train)

cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1
```

O directamente:

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
pca.n_components_
```

Curva de varianza acumulada



Para visualización se eligen 2 o 3 dimensiones; para modelado se puede tratar d como hiperparámetro.

PCA dentro de un pipeline

Cuando PCA es preprocesamiento para clasificación, el número de componentes puede ajustarse con validación cruzada.

```
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(
    PCA(random_state=42),
    RandomForestClassifier(random_state=42)
)

param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}

rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
                                random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])
```

¡Gracias!